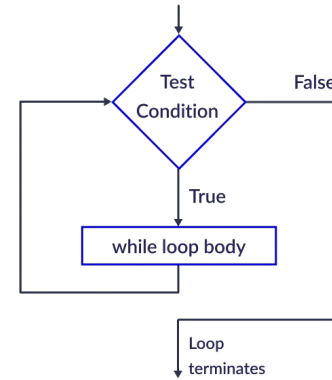
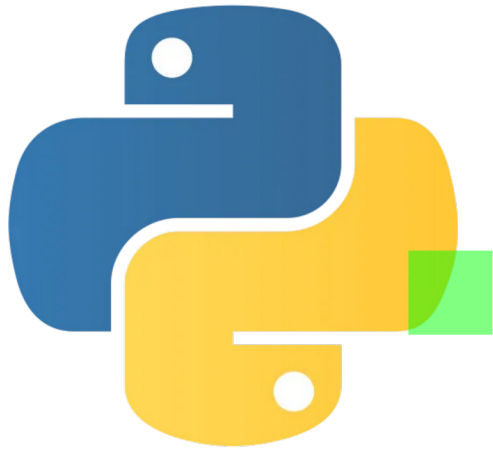


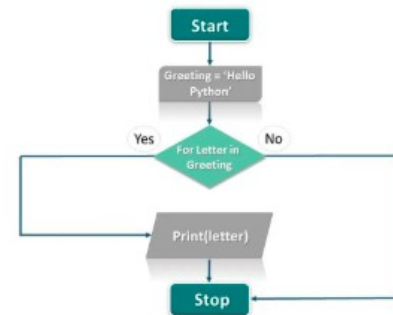


# Estruturas de Repetição (while, for) em Python

Aprenda a executar blocos de código diversas vezes de forma eficiente

# Objetivos da Aula

- ✓ Compreender o conceito de estruturas de repetição e sua importância na programação
- ✓ Dominar a sintaxe e o uso do laço while, evitando loops infinitos
- ✓ Aprender a utilizar o laço for para iteração sobre intervalos e coleções
- ✓ Entender o funcionamento da função range() e suas diferentes formas de uso
- ✓ Implementar laços aninhados para resolver problemas mais complexos
- ✓ Utilizar os comandos break e continue para controlar o fluxo de execução



# Introdução às Estruturas de Repetição

## O que são Estruturas de Repetição?

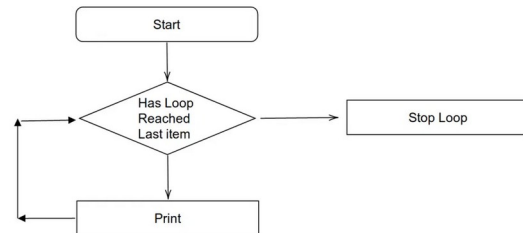
Estruturas de repetição, também conhecidas como loops, são blocos de código que permitem executar um conjunto de instruções múltiplas vezes, sem a necessidade de repetir o mesmo código.

## Por que são importantes?

- Permitem automatizar tarefas repetitivas
- Tornam o código mais conciso e legível
- Essenciais para processar coleções de dados (listas, strings, etc.)
- Fundamentais para algoritmos e solução de problemas

## Tipos principais em Python:

- while: executa enquanto uma condição for verdadeira
- for: itera sobre uma sequência de elementos



# Laço while

## Sintaxe do while

O laço **while** executa um bloco de código **enquanto** uma condição específica for **True**.

```
# Sintaxe básica
while condição:
    # Bloco de código
    # Executado enquanto a condição for True
```

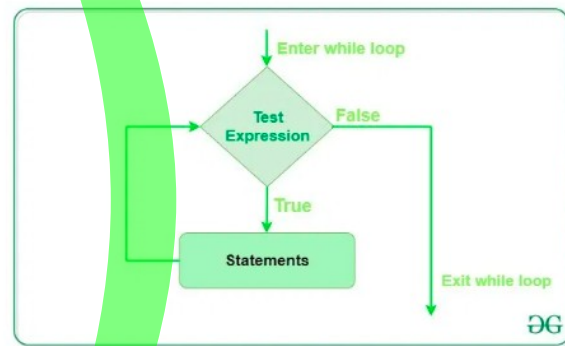
## Exemplo

```
contador = 1

while contador <= 5:
    print(f"Contagem: {contador}")
    contador += 1 # Incrementa o contador

print("Loop finalizado!")
```

**Cuidado com loops infinitos!** Sempre garanta que a condição do while eventualmente se torne False, ou use break para sair do loop.



# Laço for

## Sintaxe do for

O laço **for** em Python é usado para iterar sobre uma **sequência** (lista, tupla, dicionário, conjunto ou string).

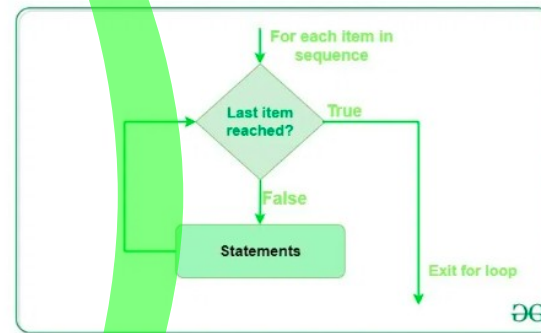
```
# Sintaxe básica
for variavel in sequencia:
    # Bloco de código a ser executado
    # para cada item da sequência
```

## Exemplos

```
# Iterando sobre uma lista
frutas = ["maçã", "banana", "laranja"]
for fruta in frutas:
    print(f"Eu gosto de {fruta}")

# Iterando sobre uma string
for letra in "Python":
    print(letra)
```

**Vantagem:** O laço for é mais simples e seguro que o while quando o número de iterações é conhecido antecipadamente.



# Função range()

## O que é a função range()?

A função `range()` gera uma sequência de números inteiros, muito útil para criar loops com um número específico de iterações.

| Sintaxe                               | Descrição                                                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>range(stop)</code>              | Gera números de 0 até <code>stop-1</code>                                                                            |
| <code>range(start, stop)</code>       | Gera números de <code>start</code> até <code>stop-1</code>                                                           |
| <code>range(start, stop, step)</code> | Gera números de <code>start</code> até <code>stop-1</code> , incrementando de <code>step</code> em <code>step</code> |

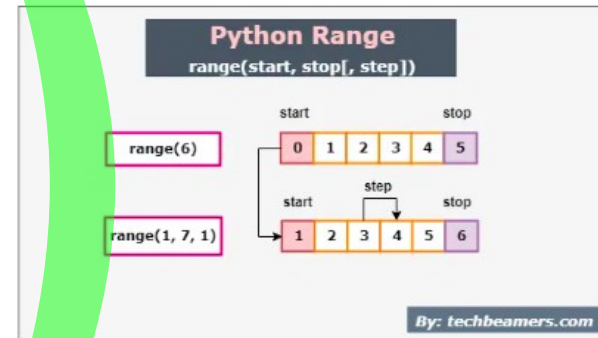
## Exemplos:

```
# Exemplo 1: range com um argumento
for i in range(5):
    print(i, end=" ")
```

0 1 2 3 4

```
# Exemplo 2: range com início e fim
for i in range(2, 8):
    print(i, end=" ")
```

2 3 4 5 6 7



# Laços Aninhados

## O que são Laços Aninhados?

Laços aninhados são **loops dentro de loops**, onde cada iteração do loop externo executa o loop interno por completo.

## Exemplo: Padrão de Asteriscos

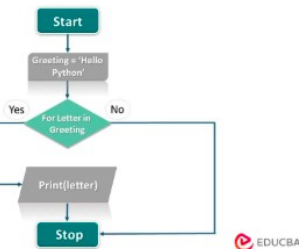
```
# Criando um padrão de asteriscos
for i in range(5): # Loop externo
    for j in range(i+1): # Loop interno
        print("*", end=" ")
    print() # Nova linha após cada linha do padrão
```

```
*
**
***
****
*****
```

## Aplicações Comuns

- Processamento de matrizes e tabelas
- Geração de padrões e formas
- Comparação de todos os elementos entre si
- Algoritmos de ordenação e busca

**Dica:** Cuidado com a eficiência! Laços aninhados têm complexidade  $O(n^2)$  ou maior, o que pode tornar o programa lento para grandes conjuntos de dados.





# Comandos break e continue

## Comando break

O comando **break** interrompe completamente a execução do laço, saindo dele imediatamente.

```
# Exemplo: Encontrar o primeiro número divisível por 7
for i in range(1, 100):
    if i % 7 == 0:
        print(f"Encontrado: {i}")
        break # Sai do loop após encontrar
```

Encontrado: 7

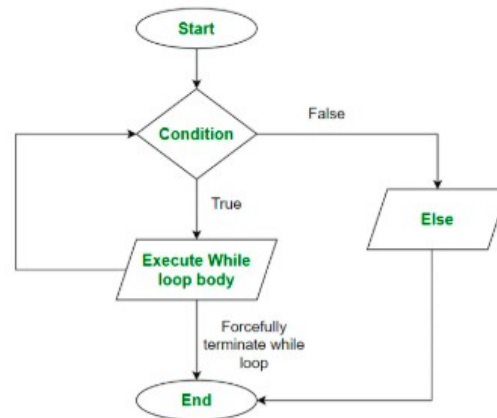
## Comando continue

O comando **continue** pula a iteração atual e avança para a próxima iteração do laço.

```
# Exemplo: Imprimir apenas números ímpares
for i in range(1, 10):
    if i % 2 == 0: # Se for par
        continue # Pula para a próxima iteração
    print(i, end=" ")
```

1 3 5 7 9

While-Else Loop Flow Chart





# Exemplos Práticos

## 1. Soma dos Números

```
# Calcular a soma dos números de 1 a 10
soma = 0
for i in range(1, 11):
    soma += i

print(f"A soma dos números de 1 a 10 é: {soma}")
```

A soma dos números de 1 a 10 é: 55

## 2. Validação de Entrada

```
# Solicitar um número entre 1 e 10
while True:
    numero = int(input("Digite um número entre 1 e 10: "))
    if 1 <= numero <= 10:
        break
    print("Valor inválido! Tente novamente.")

print(f"Você digitou: {numero}")
```

Digite um número entre 1 e 10: 15  
Valor inválido! Tente novamente.  
Digite um número entre 1 e 10: 7  
Você digitou: 7

## 3. Tabuada

```
# Gerar a tabuada de um número
numero = 5
print(f"Tabuada do {numero}:")
for i in range(1, 11):
    resultado = numero * i
    print(f"{numero} x {i} = {resultado}")
```

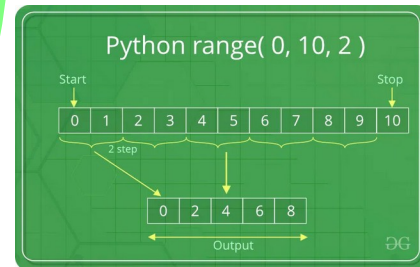
Tabuada do 5:

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

...



# Exercícios Práticos

## 1. Contagem Regressiva

Crie um programa que faça uma contagem regressiva de 10 até 1, e então imprima "Lançamento!".

```
# Use um laço for com range()
for i in range(10, 0, -1):
    # Seu código aqui
```



## 2. Soma de Números

Escreva um programa que calcule a soma de todos os números de 1 a 100 usando um laço while.

```
soma = 0
contador = 1
while contador <= 100:
    # Seu código aqui
```



## 3. Tabuada

Crie um programa que imprima a tabuada de um número fornecido pelo usuário, de 1 a 10.

```
numero = int(input("Digite um número: "))
for i in range(1, 11):
    # Seu código aqui
```

## 4. Encontrar Números Primos

Escreva um programa que encontre todos os números primos entre 1 e 50 usando laços aninhados.

```
for num in range(2, 51):
    eh_primo = True
    for i in range(2, num):
        # Verifique se num é divisível por i
```